

Praktikum Modul 3

Event, State, dan Implisit Intent pada Jetpack Compose

Nama: Ossa Amelia Zuhra Lubis

NIM: 245150600111022

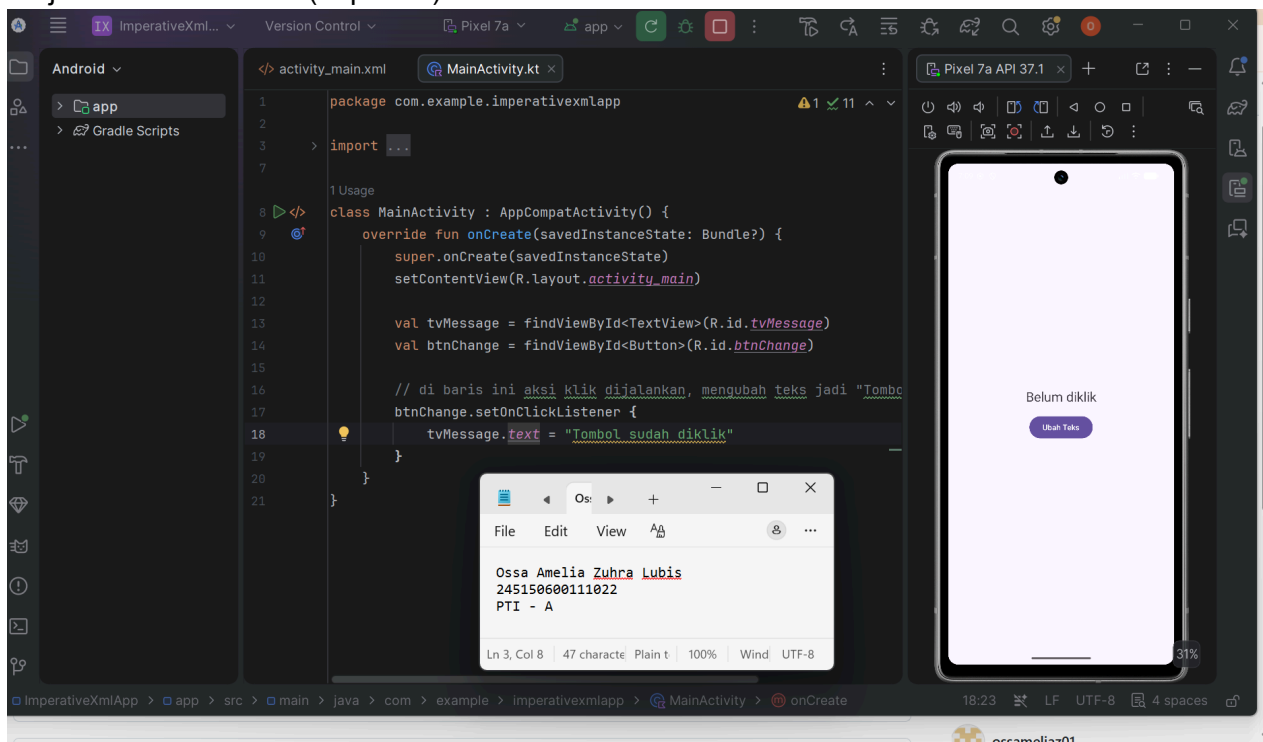
Kelas: PTI B — Fakultas Ilmu Komputer

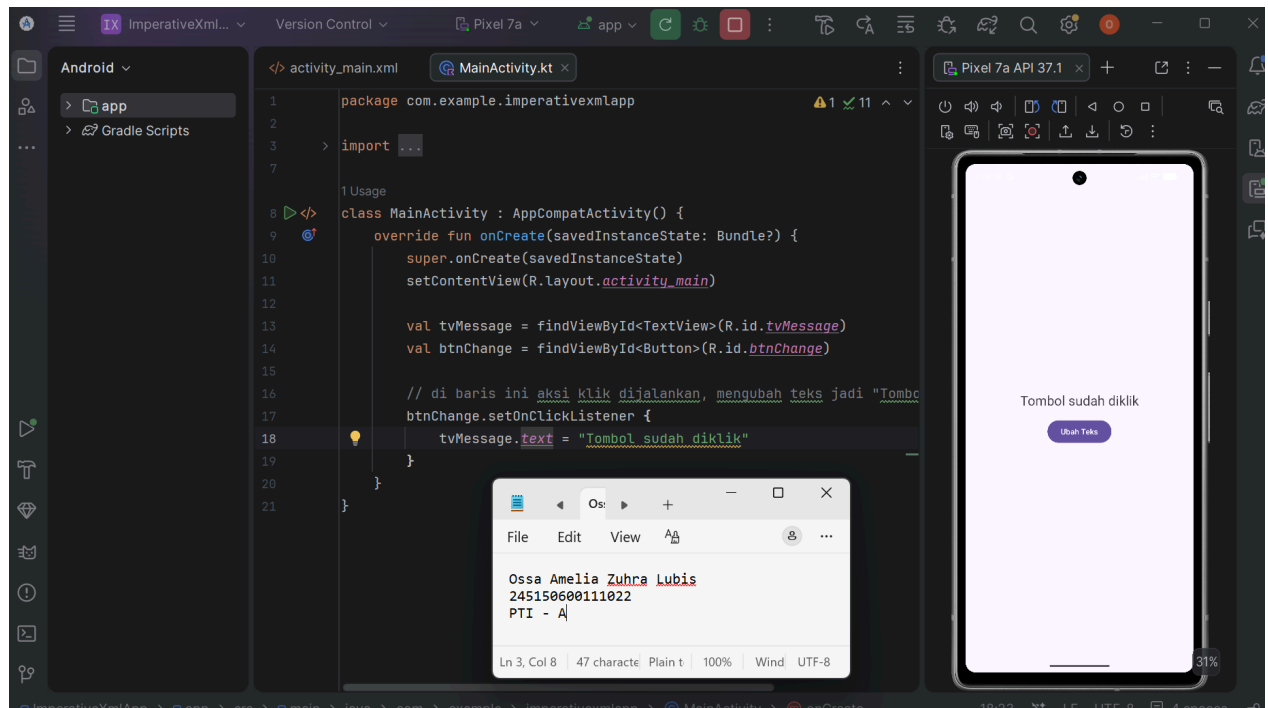
Mata Kuliah: Pengembangan Aplikasi Peranti Bergerak

Percobaan 1 — Perbandingan Imperatif dan Deklaratif

1. Jalankan kedua aplikasi, ambil screenshot kondisi sebelum dan sesudah tombol diklik, serta lampirkan kode programnya.

Project A — XML/View (Imperatif)





Code Program Project A — activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Belum diklik"
        android:textSize="24sp"
        android:layout_marginBottom="16dp" />

    <Button
        android:id="@+id/btnChange"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ubah Teks" />

</LinearLayout>
```

Code Program Project A — MainActivity.kt

```
package com.example.imperativexmlapp

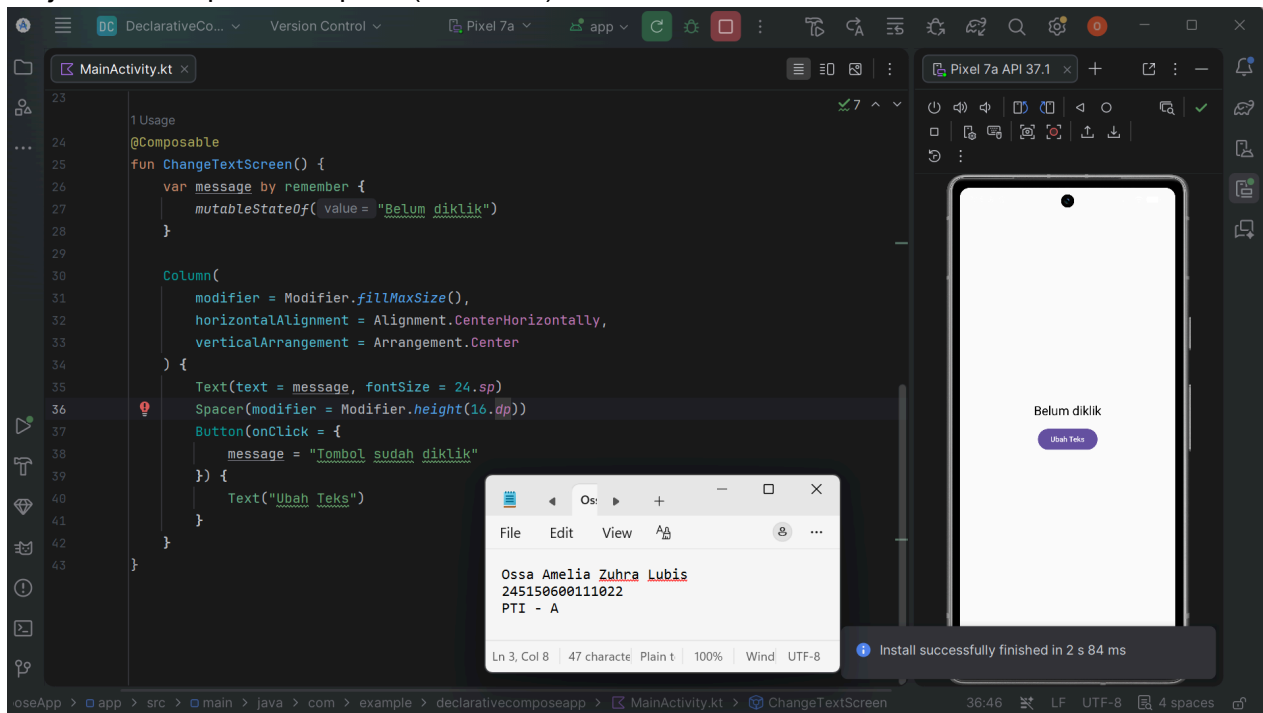
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

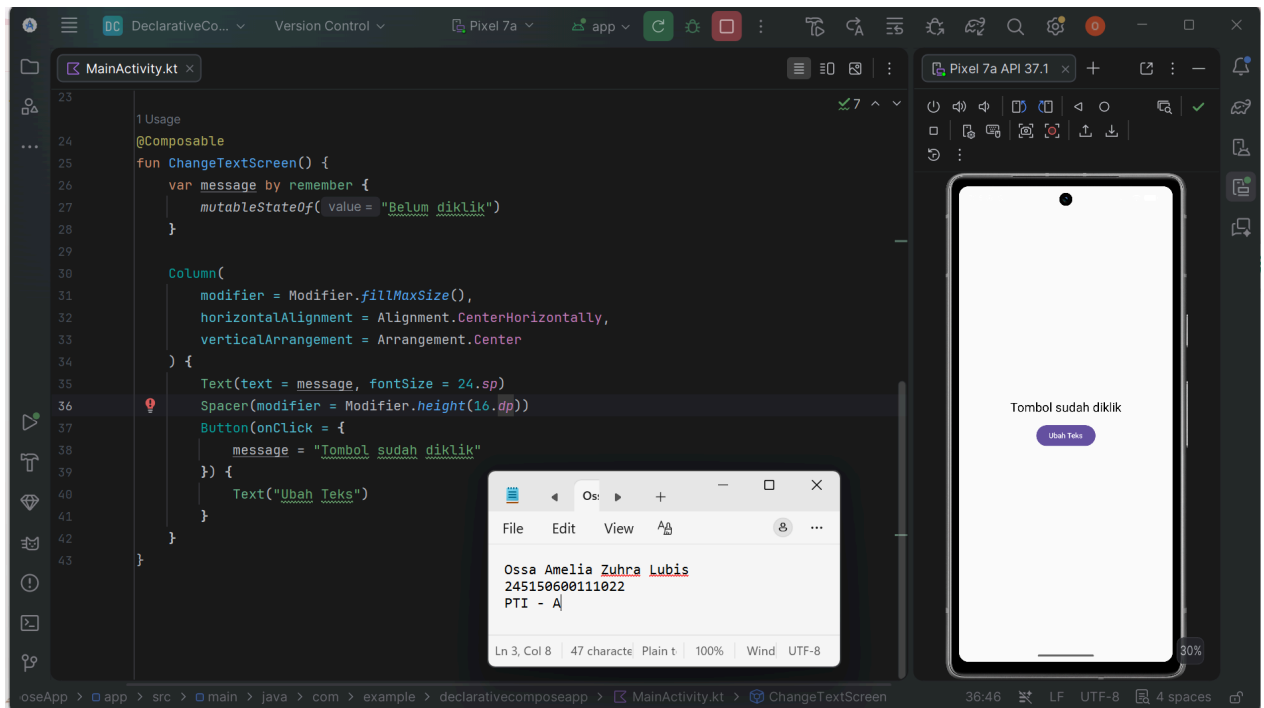
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnChange = findViewById<Button>(R.id.btnChange)

        btnChange.setOnClickListener {
            tvMessage.text = "Tombol sudah diklik"
        }
    }
}
```

Project B — Jetpack Compose (Deklaratif)





Kode Program Project B — MainActivity.kt

```
package com.example.declarativecomposeapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ChangeTextScreen()
        }
    }
}

@Composable
fun ChangeTextScreen() {
    var message by remember {
        mutableStateOf("Belum diklik")
    }
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = message, fontSize = 24.sp)
        Button(onClick = {
            message = "Tombol sudah diklik"
        }) {
            Text("Ubah Teks")
        }
    }
}
```

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawab : Objek yang diubah langsung adalah komponen TextView yang direferensikan lewat variabel tvMessage. Baris kodenya ada di dalam setOnClickListener:

```
tvMessage.text = "Tombol sudah diklik"
```

Jadi program mencari View-nya lebih dulu lewat findViewById(), baru properti (text) diubah secara langsung.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Yang diubah bukan komponen UI-nya, tapi state message. Baris kodenya ada di dalam onClick:

```
message = "Tombol sudah diklik"
```

Text(text = message) hanya membaca nilai state ini, jadi begitu message berubah, Compose otomatis menggambar ulang Text yang bergantung pada state tersebut.

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

Jawab : Karena di Compose kita tidak pernah memegang referensi langsung ke komponen UI seperti TextView. UI dianggap sebagai fungsi dari state ($UI = f(state)$), jadi yang perlu dilakukan hanyalah mengubah sumber datanya (state), lalu Compose sendiri yang bertanggung jawab menggambar ulang tampilan lewat proses recomposition. Tidak ada objek visual yang "dipegang dan diperintah" seperti pada pendekatan imperatif.

5. Jelaskan alur perubahan pada XML/View dengan pola: event → View → perubahan tampilan.

Jawab : Tombol diklik → event onClick terpicu → program mengambil referensi TextView lewat findViewById() → program mengubah properti text milik TextView tersebut secara langsung → tampilan berubah karena objek View-nya sendiri yang dimodifikasi.

6. Jelaskan alur perubahan pada Compose dengan pola: event → state → recomposition → perubahan tampilan.

Jawab : Tombol diklik → event onClick terpicu → nilai state message diubah lewat mutableStateOf → Compose mendeteksi state yang diamati (observable state) ini berubah → Compose menjalankan recomposition, yaitu menjalankan ulang fungsi composable yang bergantung pada state tersebut → Text digambar ulang dengan nilai message yang baru, sehingga tampilan berubah tanpa program pernah menyentuh objek Text secara langsung.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: var message = "Belum diklik"? Jelaskan penyebabnya.

Jawab : Kalau message jadi variabel Kotlin biasa (bukan mutableStateOf), tampilan tidak akan berubah walaupun nilainya sebenarnya berubah saat tombol diklik. Penyebabnya,

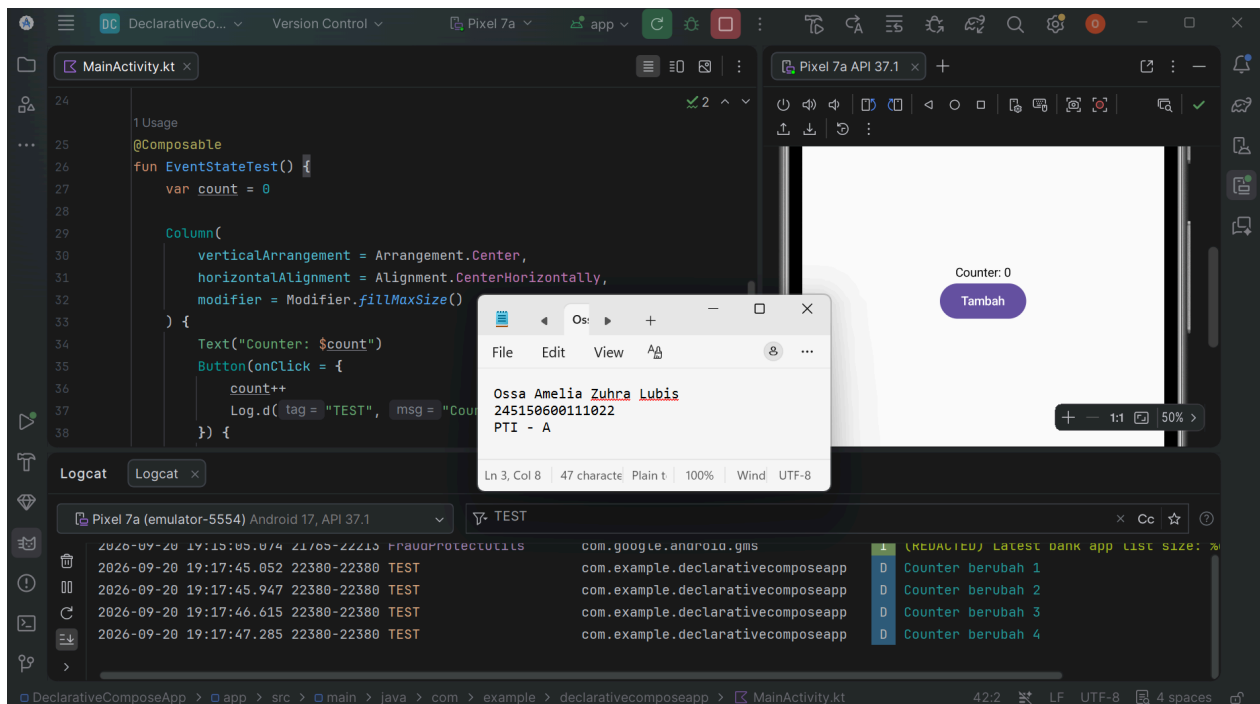
Compose hanya tahu harus melakukan recomposition kalau ada state yang "diamati" (observable) yang berubah. Variabel biasa tidak terhubung ke sistem observasi Compose, jadi perubahan nilainya tidak pernah memicu Compose untuk menggambar ulang UI. Selain itu, karena tidak dibungkus remember, nilainya juga akan selalu reset ke "Belum diklik" setiap kali composable dijalankan ulang.

8. Kesimpulan perbedaan cara berpikir imperatif dan deklaratif.

Jawab : Menurut saya, bedanya ada di siapa yang bertanggung jawab mengubah tampilan. Di pendekatan imperatif (XML/View), saya sebagai programmer yang harus mencari komponennya satu per satu lalu mengubah propertinya secara manual — jadi saya yang mengatur "bagaimana" tampilan berubah langkah demi langkah. Sedangkan di pendekatan deklaratif (Compose), saya cukup mendeskripsikan tampilan yang saya inginkan berdasarkan state saat ini, dan saya cukup mengubah state-nya saja kalau ada perubahan — soal "bagaimana" UI diperbarui itu sudah jadi tanggung jawab framework lewat recomposition. Jadi imperatif itu lebih ke "apa yang harus dilakukan", sedangkan deklaratif lebih ke "seperti apa hasil akhirnya".

Percobaan 2 — Event Handling State

1. Screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali (saat count masih var biasa), lengkap dengan LogCat dan kode program.



Kode Program EventStateTest — Tahap 1 (var count biasa)

```
@Composable
fun EventStateTest() {
    var count = 0 // TAHAP 1: masih variabel biasa (bukan state)

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```

2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik? (saat count masih var biasa)

Jawab : Log dengan tag TEST tetap muncul setiap tombol diklik, contohnya "Counter berubah : 1", "Counter berubah : 2", dan seterusnya — angkanya terus bertambah sesuai jumlah klik.

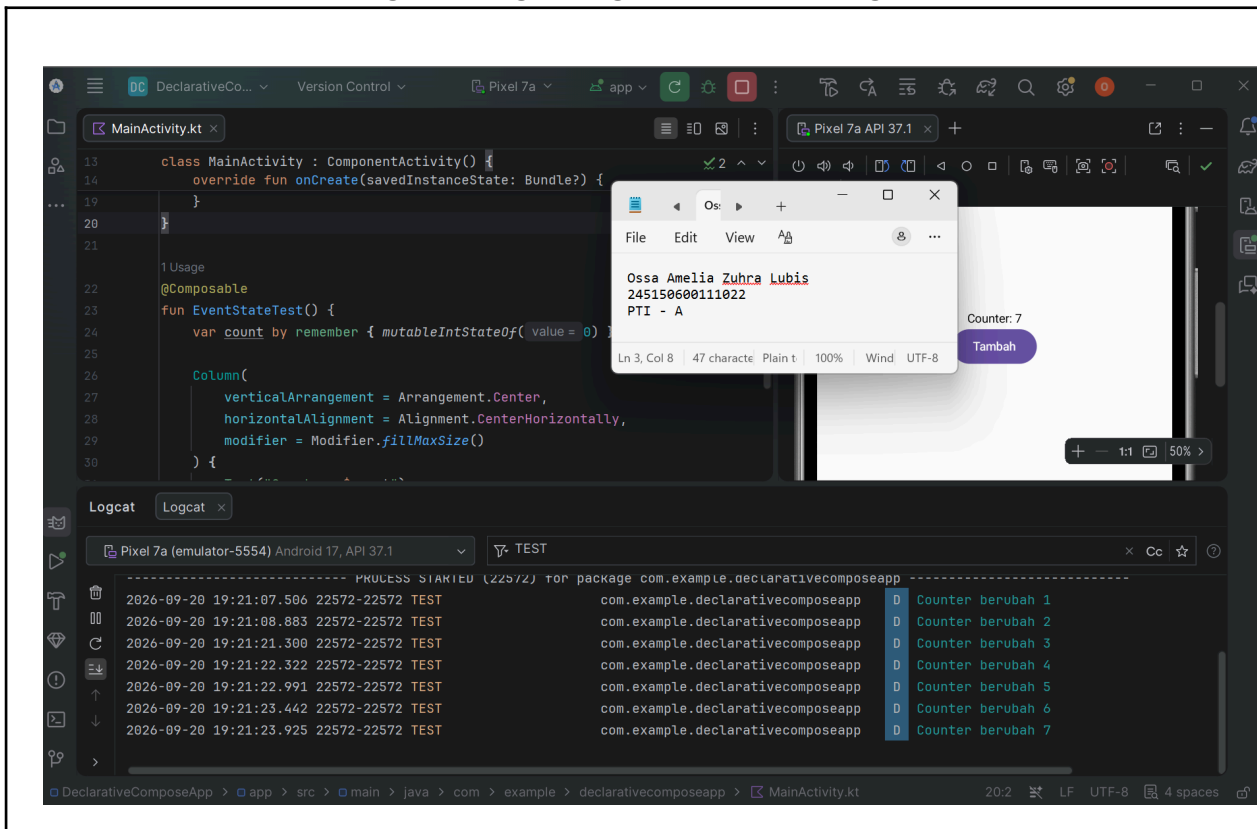
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab : Teks "Counter : 0" di layar tidak berubah sama sekali walaupun tombol diklik berkali-kali, meskipun di LogCat terlihat nilai count sebenarnya terus bertambah.

4. Apa yang menyebabkan hal tersebut terjadi?

Jawab : Karena count masih berupa variabel biasa (var count = 0), bukan state yang diamati Compose. Nilainya memang berubah di memori (makanya tetap tercatat di Log), tapi karena tidak menggunakan mutableStateOf, Compose tidak tahu bahwa ada perubahan yang perlu direspons, sehingga tidak ada recomposition dan tampilan tetap sama.

8. Screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali (setelah count memakai state), lengkap dengan LogCat dan kode program.



Kode Program EventStateTest — Tahap 2 (count pakai state)

```
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf(0) } // TAHAP 2: sudah pakai state

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```

9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik? (setelah pakai state)

Jawab : Sama seperti sebelumnya, log "Counter berubah : x" tetap muncul setiap klik dengan nilai yang terus bertambah.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab : Kali ini teks "Counter : ..." di layar ikut berubah dan bertambah setiap kali tombol diklik, sesuai dengan nilai count yang sebenarnya.

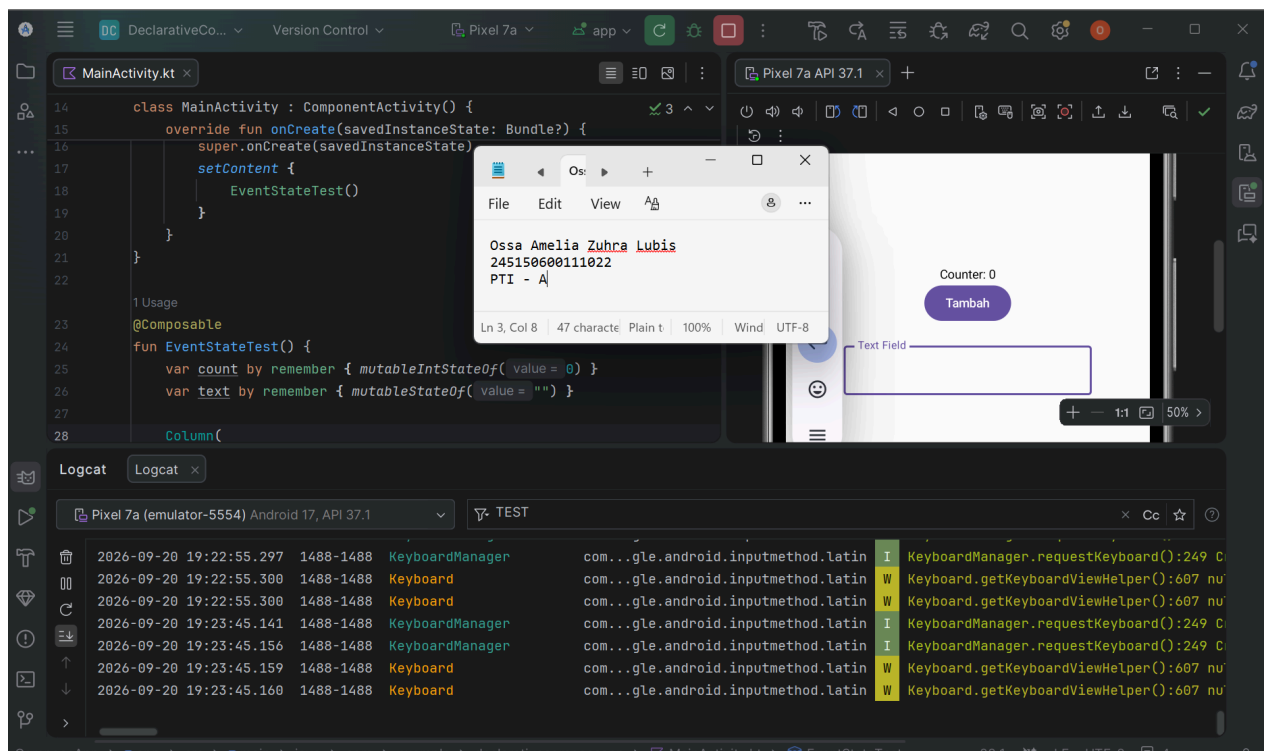
11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

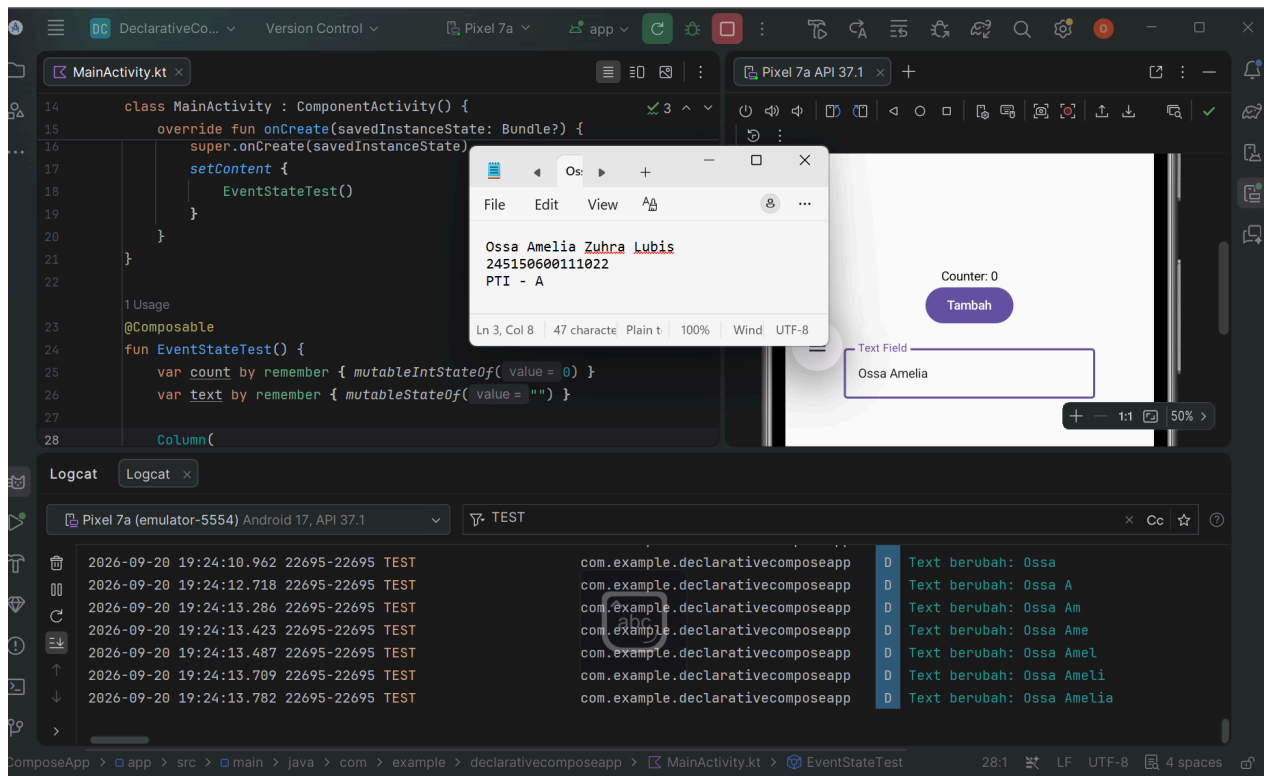
Jawab : Penyebabnya karena count sekarang dibungkus dengan `remember { mutableStateOf(0) }`, sehingga menjadi state yang diamati oleh Compose. Setiap count berubah, Compose otomatis memicu recomposition pada bagian UI yang membaca nilai count (yaitu Text), sehingga tampilan ikut ter-update. Ini berbeda dari percobaan sebelumnya karena sebelumnya count hanya variabel biasa yang tidak terhubung ke sistem observasi Compose.

12. Apa yang dimaksud recomposition pada Compose?

Jawab : Recomposition adalah proses Compose menjalankan ulang fungsi composable (atau bagian dari composable) yang bergantung pada suatu state, setiap kali state tersebut berubah nilainya. Compose cukup pintar untuk hanya menggambar ulang bagian UI yang benar-benar terpengaruh oleh perubahan state, bukan seluruh layar, sehingga prosesnya lebih efisien dibanding menggambar ulang semuanya dari awal.

14. Screenshot tampilan aplikasi setelah mengetik pada Text Field, lengkap dengan LogCat dan code program.





Kode Program EventStateTest — Tahap 3 (lengkap dengan Text Field)

```
package com.example.eventstateapp

import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            EventStateTest()
        }
    }
}

@Composable
fun EventStateTest() { // TAHAP 3: ditambah state text + OutlinedTextField
    var count by remember { mutableStateOf(0) }
    var text by remember { mutableStateOf("") }

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }

        OutlinedTextField(
            value = text,
            onValueChange = {
                text = it
                Log.d("TEST", "Text berubah : $it")
            },
            label = { Text("Text Field") }
        )
    }
}
```

15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawab : Setiap huruf yang diketik memicu log baru dengan tag TEST, misalnya "Text berubah : o", lalu "Text berubah : os", "Text berubah : oss", dan seterusnya — log ini mengikuti isi lengkap text field setelah huruf baru ditambahkan.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawab : Tampilan OutlinedTextField langsung menampilkan huruf yang baru diketik secara real-time, karakter demi karakter, sesuai apa yang diinputkan pengguna.

17. Apa yang menyebabkan hal tersebut terjadi?

Jawab : Karena `onValueChange` pada `OutlinedTextField` mengubah state `text` (`var text by remember { mutableStateOf("") }`) setiap kali ada input baru. Karena `text` adalah state yang diamati, setiap perubahan langsung memicu `recomposition` sehingga `value = text` yang ditampilkan di `TextField` selalu mengikuti input terbaru.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat.

Jawab : `count` — state `Int` untuk menyimpan jumlah klik pada tombol Tambah.

`text` — state `String` untuk menyimpan teks yang diketik pada `OutlinedTextField`.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat.

Jawab : `onClick` pada Button "Tambah".

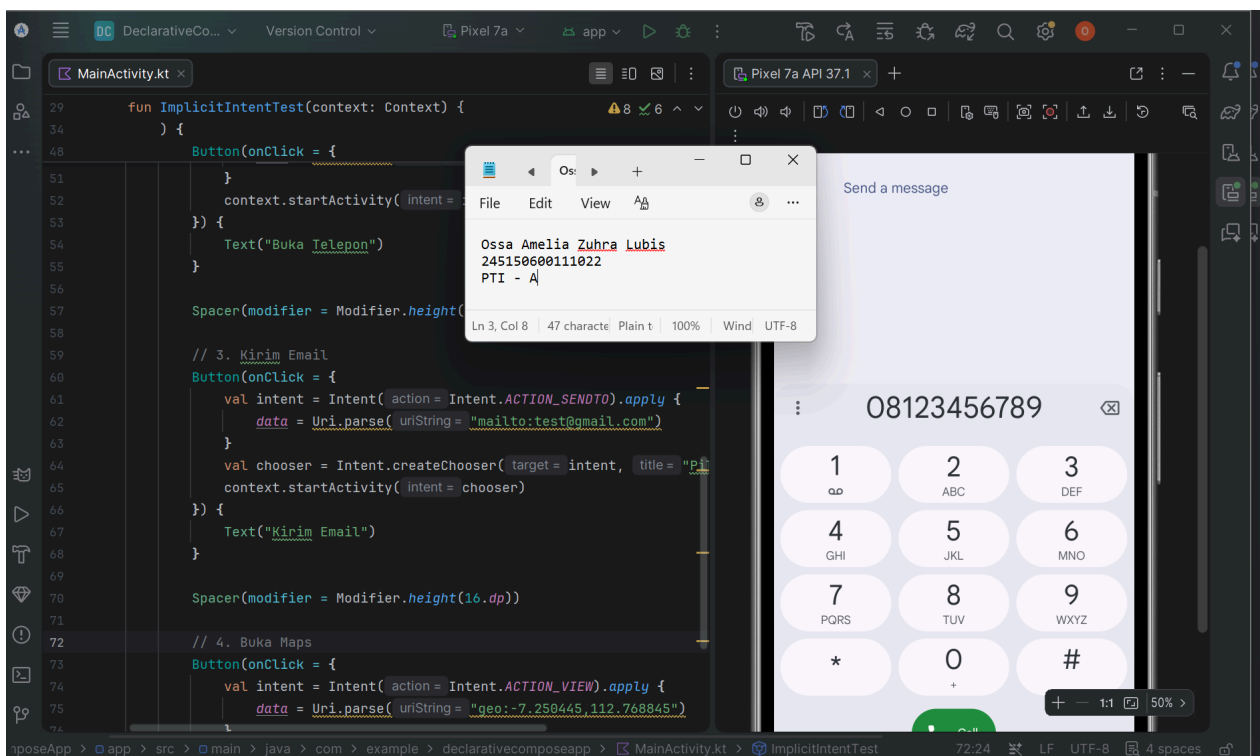
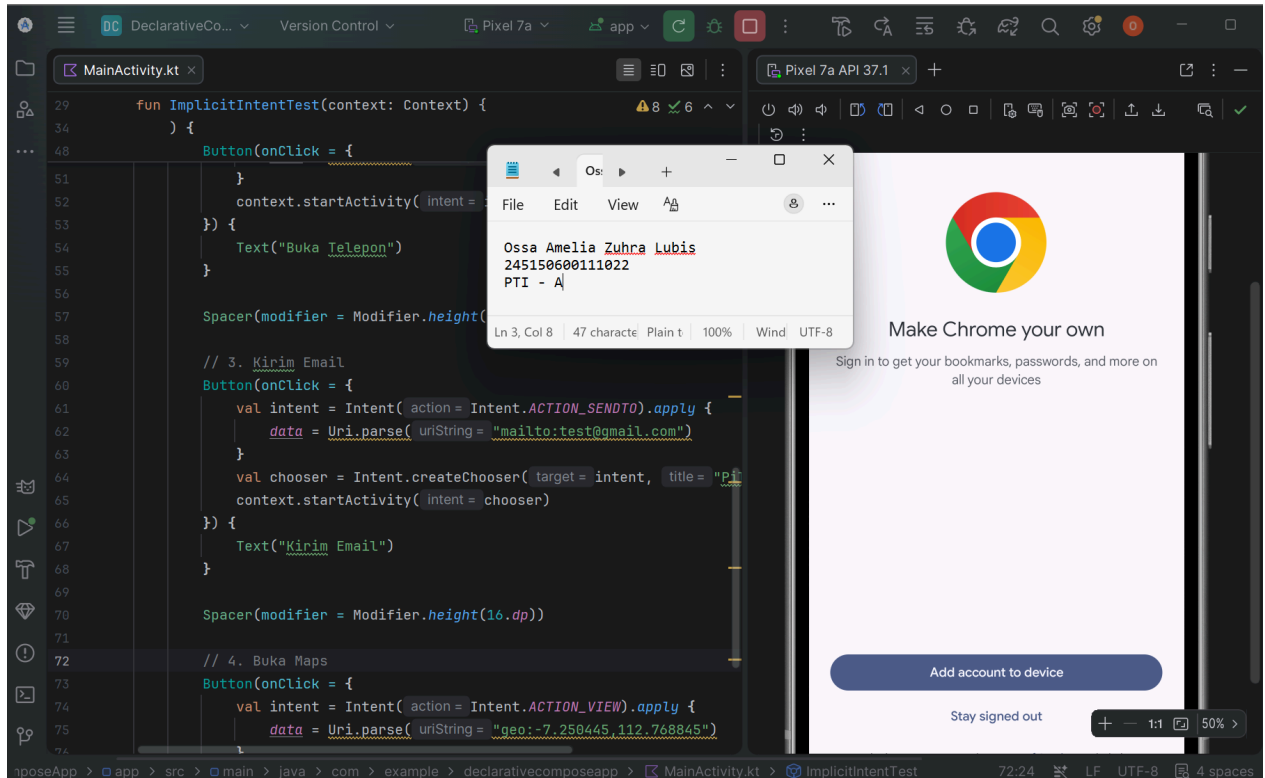
`onValueChange` pada `OutlinedTextField`.

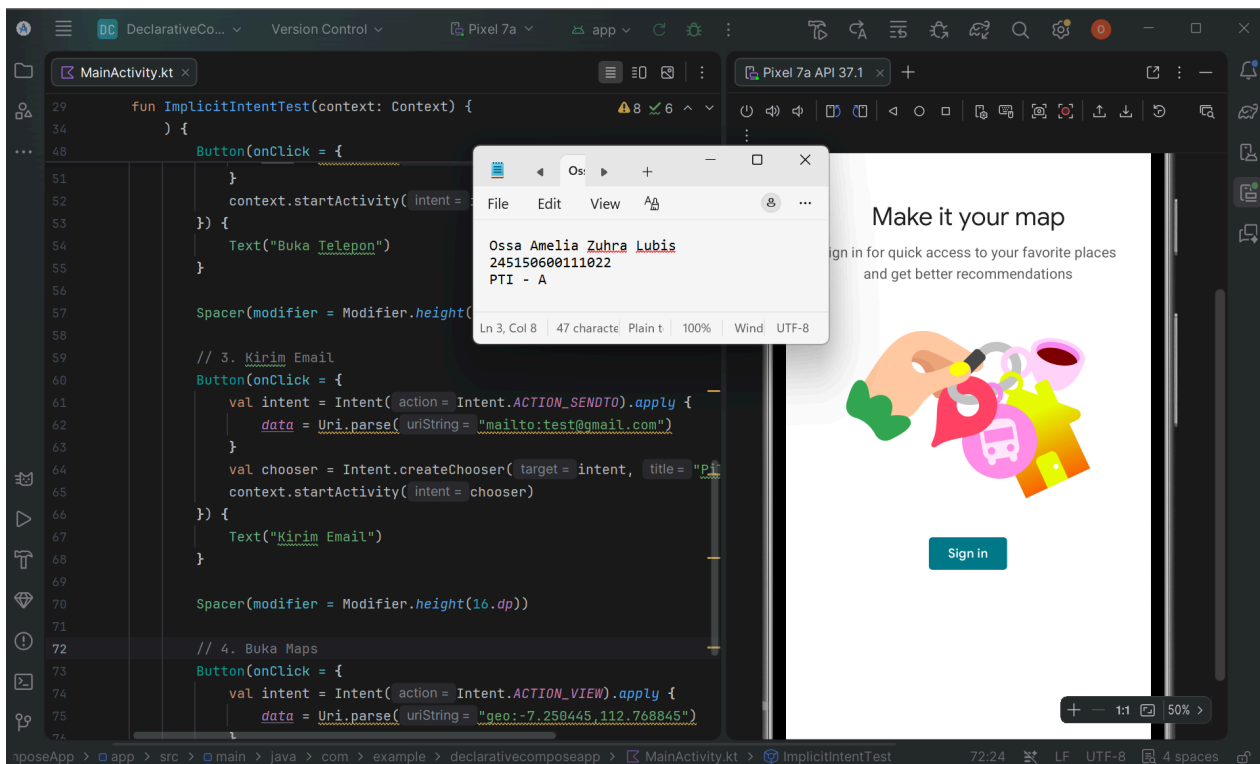
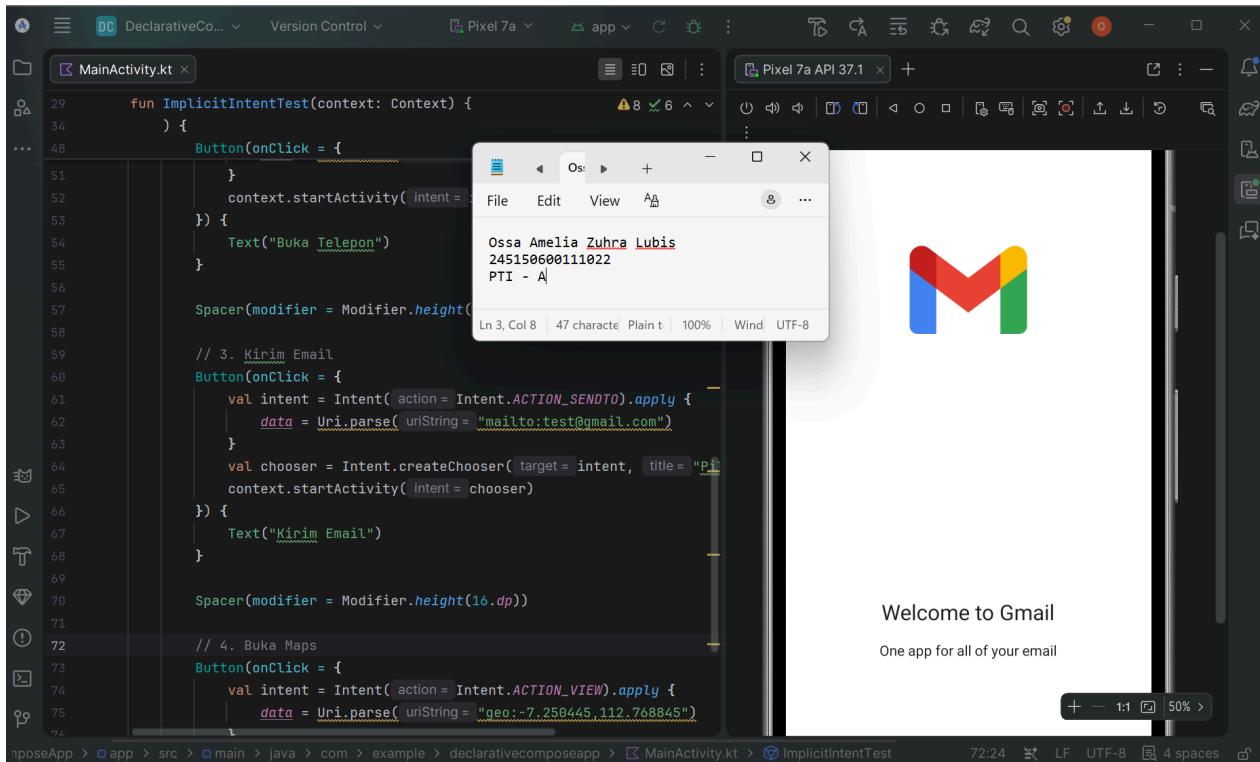
20. Kapan event-event tersebut dijalankan?

Jawab : `onClick` dijalankan setiap kali pengguna menekan/mengklik tombol Tambah. `onValueChange` dijalankan setiap kali nilai pada `OutlinedTextField` berubah, yaitu setiap pengguna mengetik, menghapus, atau menempel (`paste`) teks pada field tersebut.

Percobaan 3 — Implisit Intent

1. Jalankan program, uji setiap tombol, ambil screenshot hasilnya, serta lampirkan kode program.





Code Program ImplicitIntentTest — MainActivity.kt

```
package com.example.implicitintentapp
```

```
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
```

```

import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val context = LocalContext.current
            ImplicitIntentTest(context)
        }
    }
}

@Composable
fun ImplicitIntentTest(context: Context) {
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Button(onClick = {
            val intent = Intent(Intent.ACTION_VIEW).apply { data =
                Uri.parse("https://www.google.com")
            }
            context.startActivity(intent)
        }) { Text("Buka Browser") }

        Spacer(modifier = Modifier.height(16.dp))

        Button(onClick = {
            val intent = Intent(Intent.ACTION_DIAL).apply { data =
                Uri.parse("tel:08123456789")
            }
            context.startActivity(intent)
        }) { Text("Buka Telepon") }

        Spacer(modifier = Modifier.height(16.dp))

        Button(onClick = {
            val intent = Intent(Intent.ACTION_SENDTO).apply { data =
                Uri.parse("mailto:test@gmail.com")
            }
            val chooser = Intent.createChooser(intent, "Pilih aplikasi email")
            context.startActivity(chooser)
        }) { Text("Kirim Email") }

        Spacer(modifier = Modifier.height(16.dp))

        Button(onClick = {
            val intent = Intent(Intent.ACTION_VIEW).apply { data =
                Uri.parse("geo:-7.250445,112.768845")
            }
            context.startActivity(intent)
        }) { Text("Buka Maps") }
    }
}

```

11. Apa perbedaan fungsi dari masing-masing action intent (ACTION_VIEW, ACTION_DIAL, ACTION_SENDTO)?

Jawab : Intent.ACTION_VIEW — digunakan untuk menampilkan/membuka suatu data ke pengguna, misalnya membuka halaman website di browser atau membuka lokasi di aplikasi peta. Aksi ini bersifat umum untuk "melihat" data.

Intent.ACTION_DIAL — digunakan untuk membuka aplikasi telepon dan menampilkan nomor yang sudah terisi di layar dial, tapi tidak langsung melakukan panggilan; pengguna sendiri yang harus menekan tombol panggil.

Intent.ACTION_SENDTO — digunakan untuk mengirim pesan/data ke aplikasi tertentu berdasarkan skema data-nya, misalnya mailto: untuk membuka aplikasi email dengan

alamat tujuan yang sudah terisi.

16. Apa fungsi data pada intent?

Jawab : Data pada intent (diisi lewat Uri.parse(...)) berfungsi memberi tahu sistem Android informasi spesifik apa yang ingin diproses, dan biasanya juga menunjukkan skema/jenis data tersebut (misalnya https:// untuk web, tel: untuk nomor telepon, mailto: untuk email, geo: untuk koordinat lokasi). Skema inilah yang dipakai sistem untuk menentukan aplikasi mana yang cocok menangani intent tersebut.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Jawab : Action menjelaskan "aksi apa" yang ingin dilakukan (misalnya melihat/VIEW, mengirim/SENDTO), sedangkan Data menjelaskan "pada data seperti apa" aksi itu dilakukan (lewat skema URI-nya). Android mencocokkan kombinasi Action + skema Data ini dengan intent-filter yang didaftarkan oleh aplikasi-aplikasi yang terpasang di perangkat. Aplikasi yang intent-filter-nya cocok dengan kombinasi tersebut akan muncul sebagai pilihan untuk menangani intent — kalau ada lebih dari satu aplikasi yang cocok, sistem akan menampilkan dialog pemilihan aplikasi (chooser) kepada pengguna.